

MongoDB

MongoDB is in many ways like a power drill. Your ability to complete a task is framed largely by the components you choose to use (from drill bits of varying size to sander adapters). MongoDB's strength lies in its versatility, power, ease of use, and ability to handle jobs both large and small. Although it's a much newer invention than the hammer, it is a tool that builders reach for more and more often.

First publicly released in 2009, MongoDB (often just called *Mongo*) quickly became one of the most widely used NoSQL databases in existence, and remains somewhere very close to the top—if not right at the top—of that list today. MongoDB was designed as a scalable database—the name *Mongo* comes from “*humongous*”—with performance and easy data access as core design goals. It is a document database, which allows you to store objects nested to whichever depth you'd like, and you can query that nested data in an ad hoc fashion. It enforces no schema (similar to HBase but unlike PostgreSQL), so documents can contain fields or types that no other document in the collection contains (whether that's advisable is another matter).

But don't think that MongoDB's flexibility makes it a toy. There are some huge production MongoDB deployments out there, such as Foursquare, Comcast, Adobe, and CERN, where it's used to collect data from the Large Hadron Collider.

Hu(mongo)us

Mongo hits a sweet spot between the powerful queryability of a relational database and the distributed nature of other databases, like HBase. Project founder Dwight Merriman has said that MongoDB is the database he wishes he'd had at DoubleClick, where as the CTO he had to house large-scale data while still being able to satisfy ad hoc queries.

Mongo is a JSON document database (though technically data is stored in a binary form of JSON known as BSON). A Mongo document can be likened to a relational table row without a schema, whose values can nest to an arbitrary depth. To get an idea of what a JSON document is, check this out:

```
{
  "_id" : ObjectId("4d0b6da3bb30773266f39fea"),
  "country" : {
    "$ref" : "countries",
    "$id" : ObjectId("4d0e6074deb8995216a8309e")
  },
  "famous_for" : [
    "beer",
    "food"
  ],
  "last_census" : "Sun Jan 07 2018 00:00:00 GMT -0700 (PDT)",
  "mayor" : {
    "name" : "Ted Wheeler",
    "party" : "D"
  },
  "name" : "Portland",
  "population" : 582000,
  "state" : "OR"
}
```

} JSON document

In some ways, document databases have an opposite workflow compared to relational databases. Relational databases such as PostgreSQL assume you know what data you wish to store without necessarily knowing how you want to use it; what's important is how you store it. The cost of query flexibility is paid upfront on storage. Document databases require you to make some assumptions on how you wish to use your data, but few assumptions on what exactly you wish to store. You can make fundamental “schema” changes on-the-fly, but you may have to pay for your design decisions later on.

Mongo is an excellent choice for an ever-growing class of web projects with large-scale data storage requirements but very little budget to buy big-iron hardware. Thanks to its lack of structured schema, Mongo can grow and change along with your data model. If you're in a web startup with dreams of enormity or are already large with the need to scale servers horizontally, consider MongoDB.

Day 1: CRUD and Nesting

We'll spend today working on some CRUD operations and finish up by performing nested queries in MongoDB. As usual, we won't walk you through the installation steps, but if you visit the Mongo website,¹ you can download a build for your OS or find instructions on how to build from source. If you have OS X, we recommend installing via Homebrew (`brew install mongodb`). If you use a Debian/Ubuntu variant, try MongoDB's own `apt-get` package.

1. <https://www.mongodb.com/download-center#community>



Eric says: On the Fence

I was on the fence about using a document database until I actually spent time on teams using document stores in production. Coming from the relational database world, I found Mongo to be an easy transition with its ad hoc queries, and its ability to scale out mirrored my own web-scale dreams in ways that many relational stores couldn't. But beyond the structure, I trusted Mongo's development team. They readily admitted that Mongo wasn't perfect, but their clear plans (and general adherence to those plans) were based on general web infrastructure use cases, rather than idyllic debates on scalability and replication. This pragmatic focus on usability should shine as you use MongoDB. A trade-off of this evolutionary behavior is that there are several paths to performing any given function in Mongo.

To prevent typos, Mongo requires you to first create the directory where mongod will store its data. A common location is `/data/db`. Ensure the user you run the server under has permission to read and write to this directory. If it's not already running, you can fire up the Mongo service by running `mongod`.

Command-Line Fun

To create a new database named `book`, first run this command in your terminal. It will connect to the MySQL-inspired command-line interface.

```
$ mongo book
```

Typing `help` in the console is a good start. We're currently in the `book` database, but you can view others via `show dbs` and switch databases with the `use` command.

Creating a collection in Mongo is as easy as adding an initial record to the collection. Because Mongo is schemaless, there is no need to define anything up front; merely using it is enough. What's more, our `book` database doesn't really exist until we first add values into it. The following code creates/inserts a `towns` collection:

```
> db.towns.insert({
  name: "New York",
  population: 22200000,
  lastCensus: ISODate("2016-07-01"),
  famousFor: [ "the MOMA", "food", "Derek Jeter" ],
  mayor : {
    name : "Bill de Blasio",
    party : "D"
  }
})
```

In the previous section, we said documents were JSON (well, really BSON under the hood), so we add new documents as JSON (as we will do later on with CouchDB and, to a lesser extent, DynamoDB).

With the `show collections` command, you can verify the collection now exists.

```
> show collections
```

```
towns
```

We just created the `towns` collection by storing an object in it. We can list the contents of a collection via `find()`. We formatted the output here for readability, but yours may just output as a single wrapped line.

```
> db.towns.find()
```

```
{
  "_id" : ObjectId("59093bc08c87e2ff4157bd9f"),
  "name" : "New York",
  "population" : 22200000,
  "lastCensus" : ISODate("2016-07-01T00:00:00Z"),
  "famousFor" : [ "the MOMA", "food", "Derek Jeter" ],
  "mayor" : {
    "name" : "Bill de Blasio",
    "party" : "I"
  }
}
```

Unlike a relational database, Mongo does not support server-side joins. A single JavaScript call will retrieve a document *and* all of its nested content, free of charge.

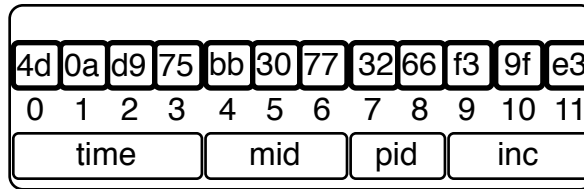
You may have noticed that the JSON output of your newly inserted town contains an `_id` field of type `ObjectId`. This is akin to `SERIAL` incrementing a numeric primary key in PostgreSQL. The `ObjectId` is always 12 bytes, composed of a timestamp, client machine ID, client process ID, and a 3-byte incremented counter. The [figure on page 97](#) shows how bytes are laid out.

What's great about this autonumbering scheme is that each process on every machine can handle its own ID generation without colliding with other mongod instances. This design choice exhibits Mongo's generally distributed nature.

JavaScript

Mongo's native tongue is JavaScript. You'll use it when doing things as complex as mapreduce queries or as simple as asking for help.

```
> db.help()
> db.towns.help()
```



These commands will list available functions related to the given object. `db` is a JavaScript object that contains information about the current database. `db.x` is a JavaScript object representing a collection (named `x`). Commands are just JavaScript functions.

```
> typeof db
object
> typeof db.towns
object
> typeof db.towns.insert
function
```

If you want to inspect the source code for a function, call it without parameters or parentheses (think more Python than Ruby).

```
> db.towns.insert
function (obj, options, _allowDot) {
  if (!obj)
    throw Error("no object passed to insert!");

  var flags = 0;

  // etc.
}
```

Let's populate a few more documents into our `towns` collection by creating our own JavaScript function.

```
mongo/insertCity.js
function insertCity(
  name, population, lastCensus,
  famousFor, mayorInfo
) {
  db.towns.insert({
    name: name,
    population: population,
    lastCensus: ISODate(lastCensus),
    famousFor: famousFor,
    mayor : mayorInfo
  });
}
```

You can just paste the code for the function into the shell. Then we can call it.

```
> insertCity("Punxsutawney", 6200, '2016-01-31',
  ["Punxsutawney Phil"], { name : "Richard Alexander" }
)
> insertCity("Portland", 582000, '2016-09-20',
  ["beer", "food", "Portlandia"], { name : "Ted Wheeler", party : "D" }
)
```

We should now have three towns in our collection, which you can confirm by calling `db.towns.find()` as before.

Mongo Data Through a Visual Lens

All of the practical exercises for Mongo in this chapter will involve accessing it either through the Mongo shell or through JavaScript code. If you're more inclined to visual representations of data—and the systems around data—you may want to explore more UI-driven tools. One very notable Mongo-specific tool is Robo 3T,^a previously known as Robomongo, which is a desktop app that enables you to visualize MongoDB datasets, monitor servers, engage in user management, edit data directly, and so on.

The authors themselves are largely disinclined toward UI-driven tools like this for databases, but Robo 3T is extremely well done, and if a nice UI brings you closer to grasping Mongo or any other database, we say go for it.

a. <https://robomongo.org/>

Reading: More Fun in Mongo

Earlier, we called the `find()` function without params to get all documents. To access a specific one, you only need to set an `_id` property. `_id` is of type `ObjectId`, and so to query, you must convert a string by wrapping it in an `ObjectId(str)` function.

```
> db.towns.find({ "_id" : ObjectId("59094288afbc9350ada6b807") })
{
  "_id" : ObjectId("59094288afbc9350ada6b807"),
  "name" : "Punxsutawney",
  "population" : 6200,
  "lastCensus" : ISODate("2016-01-31T00:00:00Z"),
  "famousFor" : [ "Punxsutawney Phil" ],
  "mayor" : { "name" : "Richard Alexander" }
}
```

The `find()` function also accepts an optional second parameter: a fields object we can use to filter which fields are retrieved. If we want only the town name (along with `_id`), pass in `name` with a value resolving to 1 (or true).

```
> db.towns.find({ _id : ObjectId("59094288afbc9350ada6b807") }, { name : 1 })
{
  "_id" : ObjectId("59093e9eafbc9350ada6b803"),
  "name" : "Punxsutawney"
}
```

To retrieve all fields *except* name, set name to 0 (or false or null).

```
> db.towns.find({ _id : ObjectId("59094288afbc9350ada6b807") }, { name : 0 })
{
  "_id" : ObjectId("59093e9eafbc9350ada6b803"),
  "population" : 6200,
  "lastCensus" : ISODate("2016-01-31T00:00:00Z"),
  "famousFor" : [ "Punxsutawney Phil" ]
}
```

As in PostgreSQL, in Mongo you can construct ad hoc queries on the basis of field values, ranges, or a combination of criteria. To find all towns that begin with the letter *P* and have a population less than 10,000, you can use a Perl-compatible regular expression (PCRE)² and a range operator. This query should return the JSON object for Punxsutawney, but including only the name and population fields:

```
> db.towns.find(
  { name : /^P/, population : { $lt : 10000 } },
  { _id: 0, name : 1, population : 1 }
)
{ "name" : "Punxsutawney", "population" : 6200 }
```

Conditional operators in Mongo follow the format of field : { \$op : value }, where \$op is an operation like \$ne (not equal to) or \$gt (greater than). You may want a terser syntax, like field < value. But this is JavaScript code, not a domain-specific query language, so queries must comply with JavaScript syntax rules (later today you'll see how to use the shorter syntax in a certain case, but we'll skip that for now).

The good news about the querying language being JavaScript is that you can construct operations as you would objects. Here, we build criteria where the population must be between 10,000 and 1 million people.

```
> var population_range = {
  $lt: 1000000,
  $gt: 10000
}
> db.towns.find(
  { name : /^P/, population : population_range },
  { name: 1 }
)
{ "_id" : ObjectId("59094292afbc9350ada6b808"), "name" : "Portland" }
```

2. <http://www.pcre.org/>

In addition to number ranges, we can also retrieve date ranges. For example, we can find all names with a *lastCensus* greater than or equal to June 1, 2016, like this:

```
> db.towns.find(
  { lastCensus : { $gte : ISODate('2016-06-01') } },
  { _id : 0, name: 1 }
)
{ "name" : "New York" }
{ "name" : "Portland" }
```

Notice how we again suppressed the *_id* field in the output explicitly by setting it to 0.

Digging Deep

Mongo loves nested array data. You can query by matching exact values:

```
> db.towns.find(
  { famousFor : 'food' },
  { _id : 0, name : 1, famousFor : 1 }
)
{ "name" : "New York", "famousFor" : [ "the MOMA", "food", "Derek Jeter" ] }
{ "name" : "Portland", "famousFor" : [ "beer", "food", "Portlandia" ] }
```

as well as matching partial values:

```
> db.towns.find(
  { famousFor : /moma/ },
  { _id : 0, name : 1, famousFor : 1 }
)
{ "name" : "New York", "famousFor" : [ "the MOMA", "food" ] }
```

or query by all matching values:

```
> db.towns.find(
  { famousFor : { $all : ['food', 'beer'] } },
  { _id : 0, name:1, famousFor:1 }
)
{ "name" : "Portland", "famousFor" : [ "beer", "food", "Portlandia" ] }
```

or the lack of matching values:

```
> db.towns.find(
  { famousFor : { $nin : ['food', 'beer'] } },
  { _id : 0, name : 1, famousFor : 1 }
)
{ "name" : "Punxsutawney", "famousFor" : [ "Punxsutawney Phil" ] }
```


But the true power of Mongo stems from its ability to dig down into a document and return the results of deeply nested subdocuments. To query a subdocument, your field name is a string separating nested layers with a dot. For instance, you can find towns with mayors from the Democratic Party:

```
> db.towns.find(
  { 'mayor.party' : 'D' },
  { _id : 0, name : 1, mayor : 1 }
)
{ "name" : "New York", "mayor" : { "name" : "Bill de Blasio", "party" : "D" } }
{ "name" : "Portland", "mayor" : { "name" : "Ted Wheeler", "party" : "D" } }
```

or those with mayors who don't have a party:

```
> db.towns.find(
  { 'mayor.party' : { $exists : false } },
  { _id : 0, name : 1, mayor : 1 }
)
{ "name" : "Punxsutawney", "mayor" : { "name" : "Richard Alexander" } }
```

The previous queries are great if you want to find documents with a single matching field, but what if you need to match several fields of a subdocument?

elemMatch

We'll round out our dig with the `$elemMatch` directive. Let's create another collection that stores countries. This time we'll override each `_id` to be a string of our choosing rather than an auto-generated identifier.

```
> db.countries.insert({
  _id : "us",
  name : "United States",
  exports : {
    foods : [
      { name : "bacon", tasty : true },
      { name : "burgers" }
    ]
  }
})
> db.countries.insert({
  _id : "ca",
  name : "Canada",
  exports : {
    foods : [
      { name : "bacon", tasty : false },
      { name : "syrup", tasty : true }
    ]
  }
})
```

```
> db.countries.insert({
  _id : "mx",
  name : "Mexico",
  exports : {
    foods : [{
      name : "salsa",
      tasty : true,
      condiment : true
    }]
  }
})
```

To validate the countries were added, we can execute the count function, expecting the number 3.

```
> db.countries.count()
3
```

Let's find a country that not only exports bacon but exports *tasty* bacon.

```
> db.countries.find(
  { 'exports.foods.name' : 'bacon', 'exports.foods.tasty' : true },
  { _id : 0, name : 1 }
)
{ "name" : "United States" }
{ "name" : "Canada" }
```

But this isn't what we wanted. Mongo returned *Canada* because it exports bacon and exports tasty syrup. `$elemMatch` helps us here. It specifies that if a document (or nested document) matches *all* of our criteria, the document counts as a match.

```
> db.countries.find(
  {
    'exports.foods' : {
      $elemMatch : {
        name : 'bacon',
        tasty : true
      }
    }
  },
  { _id : 0, name : 1 }
)
{ "name" : "United States" }
```

`$elemMatch` criteria can utilize advanced operators, too. You can find any country that exports a tasty food that also has a condiment label:

```
> db.countries.find(
  {
    'exports.foods' : {
      $elemMatch : {
        tasty : true,
        condiment : { $exists : true }
      }
    }
  },
  { _id : 0, name : 1 }
)
{ "name" : "Mexico" }
```

Mexico is just what we wanted.

Boolean Ops

So far, all of our criteria are implicitly *and* operations. If you try to find a country with the name *United States* and an *_id* of *mx*, Mongo will yield no results.

```
> db.countries.find(
  { _id : "mx", name : "United States" },
  { _id : 1 }
)
```

However, searching for one *or* the other with *\$or* will return two results. Think of this layout like *prefix notation*: OR A B.

```
db.countries.find(
  {
    $or : [
      { _id : "mx" },
      { name : "United States" }
    ]
  },
  { _id:1 }
)
{ "_id" : "us" }
{ "_id" : "mx" }
```

There are so many operators in Mongo that we can't cover them all here, but we hope this has given you a taste of MongoDB's powerful querying capabilities. The [table on page 104](#) is not a complete list of the commands but it does cover a good chunk of them.

Command	Description
\$regex	Match by any PCRE-compliant regular expression string (or just use the // delimiters as shown earlier)
\$ne	Not equal to
\$lt	Less than
\$lte	Less than or equal to
\$gt	Greater than
\$gte	Greater than or equal to
\$exists	Check for the existence of a field
\$all	Match all elements in an array
\$in	Match any elements in an array
\$nin	Does not match any elements in an array
\$elemMatch	Match all fields in an array of nested documents
\$or	or
\$nor	Not or
\$size	Match array of given size
\$mod	Modulus
\$type	Match if field is a given datatype
\$not	Negate the given operator check

You can find all the commands on the MongoDB online documentation or grab a cheat sheet from the Mongo website. We will revisit querying in the days to come.

Updating

We have a problem. New York and Punxsutawney are unique enough, but did we add Portland, Oregon, or Portland, Maine (or Texas or the others)? Let's update our towns collection to add some U.S. states.

The `update(criteria,operation)` function requires two parameters. The first is a criteria query—the same sort of object you would pass to `find()`. The second parameter is either an object whose fields will replace the matched document(s) or a modifier operation. In this case, the modifier is to `$set` the field state with the string `OR`.

```
db.towns.update(
  { _id : ObjectId("4d0ada87bb30773266f39fe5") },
  { $set : { "state" : "OR" } }
);
```

You may wonder why the \$set operation is even required. Mongo doesn't think in terms of attributes; it has only an internal, implicit understanding of attributes for optimization reasons. But nothing about the interface is *attribute-oriented*. Mongo is *document-oriented*. You will rarely want something like this (notice the lack of \$set operation):

```
db.towns.update(
  { _id : ObjectId("4d0ada87bb30773266f39fe5") },
  { state : "OR" }
);
```

This would replace the *entire* matching document with the document you gave it ({ state: "OR" }). Because you didn't give it a command like \$set, Mongo assumes you just want to switch them up, so be careful.

We can verify our update was successful by finding it (note our use of findOne() to retrieve only one matching object).

```
db.towns.findOne({ _id : ObjectId("4d0ada87bb30773266f39fe5") })

{
  "_id" : ObjectId("4d0ada87bb30773266f39fe5"),
  "famousFor" : [
    "beer",
    "food",
    "Portlandia"
  ],
  "lastCensus" : "Thu Sep 20 2017 00:00:00 GMT-0700 (PDT)",
  "mayor" : {
    "name" : "Sam Adams",
    "party" : "D"
  },
  "name" : "Portland",
  "population" : 582000,
  "state" : "OR"
}
```

You can do more than \$set a value. \$inc (increment a number) is a pretty useful one. Let's increment Portland's population by 1,000.

```
db.towns.update(
  { _id : ObjectId("4d0ada87bb30773266f39fe5") },
  { $inc : { population : 1000 } }
)
```

There are more directives than this, such as the \$ positional operator for arrays. New operations are added frequently and are updated in the online documentation. The [list on page 106](#) includes the major directives.

Command	Description
\$set	Sets the given field with the given value
\$unset	Removes the field
\$inc	Adds the given field by the given number
\$pop	Removes the last (or first) element from an array
\$push	Adds the value to an array
\$pushAll	Adds all values to an array
\$addToSet	Similar to push, but won't duplicate values
\$pull	Removes matching values from an array
\$pullAll	Removes all matching values from an array

References

As we mentioned previously, Mongo isn't built to perform joins. Because of its distributed nature, joins in Mongo would be pretty inefficient operations. Still, it's sometimes useful for documents to reference each other. In these cases, the Mongo community suggests that you use a construct like { \$ref : "collection_name", \$id : "reference_id" }. For example, we can update the towns collection to contain a reference to a document in countries.

```
> db.towns.update(
  { _id : ObjectId("59094292afbc9350ada6b808") },
  { $set : { country: { $ref: "countries", $id: "us" } } }
)
```

Now you can retrieve Portland from your towns collection.

```
> var portland = db.towns.findOne(
  { _id : ObjectId("59094292afbc9350ada6b808") }
)
```

Then, to retrieve the town's country, you can query the countries collection using the stored \$id.

```
> db.countries.findOne({ _id: portland.country.$id })
```

Better yet, in JavaScript, you can ask the town document the name of the collection stored in the fields reference.

```
> var portlandCountryRef = portland.country.$ref;
> db[portlandCountryRef].findOne({ _id: portland.country.$id })
```

The last two queries are equivalent; the second is just a bit more data-driven.

Spelling Bee Warning

Mongo, and schemaless databases in general, are not very friendly when it comes to misspellings. If you haven't run across this problem yet, you probably will at some point, so be warned. You can draw parallels between static and dynamic programming languages. You define static up front, while dynamic will accept values you may not have intended, even nonsensical types like `person_name = 5`.

Documents are schemaless, so Mongo has no way of knowing if you intended to insert pipulation into your city or meant to query on `lust_census`; it will happily insert those fields or return no matching values. This can get you in trouble later on when you try to find a document that matches the condition `population > 10000` and the result set is incomplete because Mongo doesn't even know that the object was intended to have a `population` field.

This is less of a problem when you use Mongo in a more programmatic and less ad-hoc way, as we're doing here. But keep in mind that flexibility has its price. *Caveat emptor*.

Deleting

Removing documents from a collection is simple. Just replace the `find()` function with a call to `remove()`, and all documents that match given the criteria will be removed. It's important to note that the entire matching document will be removed, not just a matching element or a matching subdocument.

We recommend running `find()` to verify your criteria before running `remove()`. Mongo won't think twice before running your operation. Let's remove all countries that export bacon that isn't tasty.

```
> var badBacon = {
  'exports.foods' : {
    $elemMatch : {
      name : 'bacon',
      tasty : false
    }
  }
}
> db.countries.find(badBacon)
{
  "_id" : ObjectId("4d0b7b84bb30773266f39fef"),
  "name" : "Canada",
  "exports" : {
    "foods" : [
      {
        "name" : "bacon",
        "tasty" : false
      }
    ]
  },
}
```

```

    {
      "name" : "syrup",
      "tasty" : true
    }
  ]
}

```

Everything looks good. Let's remove it.

```

> db.countries.remove(badBacon)
> db.countries.count()
2

```

Now when you run `count()`, verify we are left with only two countries. If so, our parameter-targeted delete was successful!

Reading with Code

Let's close out this day with one more interesting query option: `code`. You can request that MongoDB run a decision function across your documents. We placed this last because it should always be a last resort. These queries run quite slowly, you can't index them, and Mongo can't optimize them. But sometimes it's hard to beat the power of custom code.

Let's say that we're looking for a city with a population between 6,000 and 600,000 people.

```

> db.towns.find(function() {
  return this.population > 6000 && this.population < 600000;
})

```

That should return Portland and Punxsutawney. Mongo even has a shortcut for simple decision functions.

```

> db.towns.find("this.population > 6000 && this.population < 600000")

```

You can run custom code with other criteria using the `$where` clause. In this example, the query also filters for towns famous for groundhogs named Phil.

```

db.towns.find({
  $where: "this.population > 6000 && this.population < 600000",
  famousFor: /Phil/
})

```

A word of warning: Mongo will blindly run this function against each document despite there being no guarantee that the given field exists in every document. For example, if you assume a `population` field exists and `population` is missing in even a *single* document, the entire query will fail because the JavaScript cannot properly execute. Be careful when you write custom JavaScript

functions, be comfortable using JavaScript before attempting custom code, and in general avoid these sorts of operations in production.

Day 1 Wrap-Up

Today we took a peek at our first document database, MongoDB. We saw how we can store nested structured data as JSON objects and query that data to any depth. You learned that a *document* can be envisioned as a schemaless row in the relational model, keyed by a generated `_id`. A set of documents is called a *collection* in Mongo, similar to a *table* in PostgreSQL but also quite different.

Unlike the previous styles we've encountered, with collections of sets of simple datatypes, Mongo stores complex, denormalized documents, stored and retrieved as collections of arbitrary JSON structures. Mongo tops off this flexible storage strategy with a powerful query mechanism unconstrained by any predefined schema.

Its denormalized nature makes a document database a superb choice for storing data with unknown qualities, while other styles (such as relational or columnar) prefer, or sometimes even demand, that you know your data models in advance and require schema migrations to add or edit fields.

Day 1 Homework

Find

1. Bookmark the online MongoDB documentation and read up on something you found intriguing today.
2. Look up how to construct regular expressions in Mongo.
3. Acquaint yourself with command-line `db.help()` and `db.collections.help()` output.
4. Find a Mongo driver in your programming language of choice (Ruby, Java, PHP, Go, Elixir, and so on).

Do

1. Print a JSON document containing `{ "hello" : "world" }`.
2. Select a town via a case-insensitive regular expression containing the word *new*.
3. Find all cities whose names contain an *e* and are famous for food or beer.
4. Create a new database named *blogger* with a collection named *articles*. Insert a new article with an author name and email, creation date, and text.

5. Update the article with an array of comments, containing a comment with an author and text.
6. Run a query from an external JavaScript file that you create yourself.

Day 2: Indexing, Aggregating, Mapreduce

Increasing MongoDB's query performance is the first item on today's docket, followed by some more powerful and complex grouped queries. Finally, we'll round out the day with some data analysis using mapreduce.

Indexing: When Fast Isn't Fast Enough

One of Mongo's useful built-in features is indexing in the name of enhanced query performance—something, as you've seen, that's not available on all NoSQL databases. MongoDB provides several of the best data structures for indexing, such as the classic B-tree as well as other additions, such as two-dimensional and spherical GeoSpatial indexes.

For now, we're going to do a little experiment to see the power of MongoDB's B-tree index by populating a series of phone numbers with a random country prefix (feel free to replace this code with your own country code). Enter the following code into your console. This will generate 100,000 phone numbers (it may take a while), between *1-800-555-0000* and *1-800-565-0000*.

`mongo/populatePhones.js`

```
populatePhones = function(area, start, stop) {
  for(var i = start; i < stop; i++) {
    var country = 1 + ((Math.random() * 8) << 0);
    var num = (country * 1e10) + (area * 1e7) + i;
    var fullNumber = "+" + country + " " + area + "-" + i;
    db.phones.insert({
      _id: num,
      components: {
        country: country,
        area: area,
        prefix: (i * 1e-4) << 0,
        number: i
      },
      display: fullNumber
    });
    print("Inserted number " + fullNumber);
  }
  print("Done!");
}
```

Run the function with a three-digit area code (like 800) and a range of seven-digit numbers (5,550,000 to 5,650,000—please verify your zeros when typing).

```
> populatePhones(800, 5550000, 5650000) // This could take a minute
> db.phones.find().limit(2)

{ "_id" : 18005550000, "components" : { "country" : 1, "area" : 800,
  "prefix" : 555, "number" : 5550000 }, "display" : "+1 800-5550000" }
{ "_id" : 88005550001, "components" : { "country" : 8, "area" : 800,
  "prefix" : 555, "number" : 5550001 }, "display" : "+8 800-5550001" }
```

Whenever a new collection is created, Mongo automatically creates an index by the `_id`. These indexes can be found in the `system.indexes` collection. The following query shows all indexes in the database:

```
> db.getCollectionNames().forEach(function(collection) {
  print("Indexes for the " + collection + " collection:");
  printjson(db[collection].getIndexes());
});
```

Most queries will include more fields than just the `_id`, so we need to make indexes on those fields.

We're going to create a B-tree index on the `display` field. But first, let's verify that the index will improve speed. To do this, we'll first check a query without an index. The `explain()` method is used to output details of a given operation.

```
> db.phones.find({display: "+1 800-5650001"}).
  explain("executionStats").executionStats
{
  "executionTimeMillis": 52,
  "executionStages": {
    "executionTimeMillisEstimate": 58,
  }
}
```

Your output will differ from ours here and only a few fields from the output are shown here, but note the `executionTimeMillisEstimate` field—milliseconds to complete the query—will likely be double digits.

We create an index by calling `ensureIndex(fields,options)` on the collection. The `fields` parameter is an object containing the fields to be indexed against. The `options` parameter describes the type of index to make. In this case, we're building a unique index on `display` that should just drop duplicate entries.

```
> db.phones.ensureIndex(
  { display : 1 },
  { unique : true, dropDups : true }
)
```

Now try `find()` again, and check `explain()` to see whether the situation improves.

```
> db.phones.find({ display: "+1 800-5650001" }).
  explain("executionStats").executionStats
{
  "executionTimeMillis" : 0,
  "executionStages": {
    "executionTimeMillisEstimate": 0,
  }
}
```

The `executionTimeMillisEstimate` changed from 52 to 0—an infinite improvement (52 / 0)! Just kidding, but the query is now orders of magnitude faster. Mongo is no longer doing a full collection scan but instead walking the tree to retrieve the value. Importantly, scanned objects dropped from 109999 to 1—since it has become a single unique lookup.

`explain()` is a useful function, but you'll use it only when testing specific query calls. If you need to profile in a normal test or production environment, you'll need the *system profiler*.

Let's set the profiling level to 2 (level 2 stores all queries; profiling level 1 stores only slower queries greater than 100 milliseconds) and then run `find()` as normal.

```
> db.setProfilingLevel(2)
> db.phones.find({ display : "+1 800-5650001" })
```

This will create a new object in the `system.profile` collection, which you can read as any other table to get information about the query, such as a timestamp for when it took place and performance information (such as `executionTimeMillisEstimate` as shown). You can fetch documents from that collection like any other:

```
> db.system.profile.find()
```

This will return a list of objects representing past queries. This query, for example, would return stats about execution times from the first query in the list:

```
> db.system.profile.find()[0].execStats
{
  "stage" : "EOF",
  "nReturned" : 0,
  "executionTimeMillisEstimate" : 0,
  "works" : 0,
  "advanced" : 0,
  "needTime" : 0,
```

```

    "needYield" : 0,
    "saveState" : 0,
    "restoreState" : 0,
    "isEOF" : 1,
    "invalidates" : 0
  }
}

```

Like yesterday's nested queries, Mongo can build your index on nested values. If you wanted to index on all area codes, use the dot-notated field representation: `components.area`. In production, you should always build indexes in the background using the `{ background : 1 }` option.

```
> db.phones.ensureIndex({ "components.area": 1 }, { background : 1 })
```

If we find() all of the system indexes for our phones collection, the new one should appear last. The first index is always automatically created to quickly look up by `_id`, and the other two we added ourselves.

```

> db.phones.getIndexes()
[
  {
    "v" : 2,
    "key" : {
      "_id" : 1
    },
    "name" : "_id_",
    "ns" : "book.phones"
  },
  {
    "v" : 2,
    "unique" : true,
    "key" : {
      "display" : 1
    },
    "name" : "display_1",
    "ns" : "book.phones"
  },
  {
    "v" : 2,
    "key" : {
      "components.area" : 1
    },
    "name" : "components.area_1",
    "ns" : "book.phones",
    "background" : 1
  }
]

```

Our `book.phones` indexes have rounded out quite nicely.

We should close this section by noting that creating an index on a large collection can be slow and resource-intensive. Indexes simply “cost” more in Mongo than in a relational database like Postgres due to Mongo’s schemaless nature. You should always consider these impacts when building an index by creating indexes at off-peak times, running index creation in the background, and running them manually rather than using automated index creation. There are plenty more indexing tricks and tips online, but these are the basics that may come in handy the most often.

Mongo’s Many Useful CLI Tools

Before we move on to aggregation in Mongo, we want to briefly tell you about the other shell goodies that Mongo provides out-of-the-box in addition to `mongod` and `mongo`. We won’t cover them in this book but we do strongly recommend checking them out, as they together make up one of the most amply equipped CLI toolbelts in the NoSQL universe.

Command	Description
<code>mongodump</code>	Exports data from Mongo into <code>.bson</code> files. That can mean entire collections or databases, filtered results based on a supplied query, and more.
<code>mongofiles</code>	Manipulates large GridFS data files (GridFS is a specification for BSON files exceeding 16 MB).
<code>mongooplog</code>	Polls operation logs from MongoDB replication operations.
<code>mongorestore</code>	Restores MongoDB databases and collections from backups created using <code>mongodump</code> .
<code>mongostat</code>	Displays basic MongoDB server stats.
<code>mongoexport</code>	Exports data from Mongo into CSV (comma-separated value) and JSON files. As with <code>mongodump</code> , that can mean entire databases and collections or just some data chosen on the basis of query parameters.
<code>mongoimport</code>	Imports data into Mongo from JSON, CSV, or TSV (term-separated value) files. We’ll use this tool on Day 3.
<code>mongoperf</code>	Performs user-defined performance tests against a MongoDB server.
<code>mongos</code>	Short for “MongoDB shard,” this tool provides a service for properly routing data into a sharded MongoDB cluster (which we will not cover in this chapter).
<code>mongotop</code>	Displays usage stats for each collection stored in a Mongo database.
<code>bsondump</code>	Converts BSON files into other formats, such as JSON.

For more in-depth info, see the MongoDB reference documentation.^a

a. <https://docs.mongodb.com/manual/reference/program>

Aggregated Queries

MongoDB includes a handful of single-purpose *aggregators*: `count()` provides the number of documents included in a result set (which we saw earlier), `distinct()` collects the result set into an array of unique results, and `aggregate()` returns documents according to a logic that you provide.

The queries we investigated yesterday were useful for basic data extraction, but any post-processing would be up to you to handle. For example, say you wanted to count the phone numbers greater than 5599999 or provide nuanced data about phone number distribution in different countries—in other words, to produce aggregate results using many documents. As in PostgreSQL, `count()` is the most basic aggregator. It takes a query and returns a number (of matching documents).

```
> db.phones.count({'components.number': { $gt : 5599999 } })
50000
```

The `distinct()` method returns each matching value (not a full document) where one or more exists. We can get the distinct component numbers that are less than 5,550,005 in this way:

```
> db.phones.distinct('components.number',
  {'components.number': { $lt : 5550005 } })
[ 5550000, 5550001, 5550002, 5550003, 5550004 ]
```

The `aggregate()` method is more complex but also much more powerful. It enables you to specify a pipeline-style logic consisting of *stages* such as: `$match` filters that return specific sets of documents; `$group` functions that group based on some attribute; a `$sort()` logic that orders the documents by a sort key; and many others.³

You can chain together as many stages as you'd like, mixing and matching at will. Think of `aggregate()` as a combination of WHERE, GROUP BY, and ORDER BY clauses in SQL. The analogy isn't perfect, but the aggregation API does a lot of the same things.

Let's load some city data into Mongo. There's an included `mongoCities100000.js` file containing insert statements for data about nearly 100,000 cities. Here's how you can execute that file in the Mongo shell: c

```
> load('mongoCities100000.js')
> db.cities.count()
99838
```

3. <https://docs.mongodb.com/manual/reference/operator/aggregation-pipeline/>

Here's an example document for a city:

```
{
  "_id" : ObjectId("5913ec4c059c950f9b799895"),
  "name" : "Sant Julià de Lòria",
  "country" : "AD",
  "timezone" : "Europe/Andorra",
  "population" : 8022,
  "location" : {
    "longitude" : 42.46372,
    "latitude" : 1.49129
  }
}
```

We could use `aggregate()` to, for example, find the average population for all cities in the Europe/London timezone. To do so, we could `$match` all documents where `timezone` equals Europe/London, and then add a `$group` stage that produces one document with an `_id` field with a value of `averagePopulation` and an `avgPop` field that displays the average value across all population values in the collection:

```
> db.cities.aggregate([
  {
    $match: {
      'timezone': {
        $eq: 'Europe/London'
      }
    }
  },
  {
    $group: {
      _id: 'averagePopulation',
      avgPop: {
        $avg: '$population'
      }
    }
  }
])
{ "_id" : "averagePopulation", "avgPop" : 23226.22149712092 }
```

We could also match all documents in that same timezone, sort them in descending order by population, and then `$project` documents that only contain the population field:

```
> db.cities.aggregate([
  {
    // same $match statement the previous aggregation operation
  },
```



```

{
  $sort: {
    population: -1
  }
},
{
  $project: {
    _id: 0,
    name: 1,
    population: 1
  }
}
])

```

You should see results like this:

```

{ "name" : "City of London", "population" : 7556900 }
{ "name" : "London", "population" : 7556900 }
{ "name" : "Birmingham", "population" : 984333 }
// many others

```

Experiment with it a bit—try combining some of the stage types we’ve already covered in new ways—and then delete the collection when you’re done, as we’ll add the same data back into the database using a different method on Day 3.

```
> db.cities.drop()
```

This provides a very small taste of Mongo’s aggregation capabilities. The possibilities are really endless, and we encourage you to explore other stage types. Be forewarned that aggregations *can* be quite slow if you add a lot of stages and/or perform them on very large collections. There are limits to how well Mongo, as a schemaless database, can optimize these sorts of operations. But if you’re careful to keep your collections reasonably sized and, even better, structure your data to not require bold transformations to get the outputs you want, then `aggregate()` can be a powerful and even speedy tool.

Server-Side Commands

In addition to evaluating JavaScript functions, there are several pre-built commands in Mongo, most of which are executed on the server, although some require executing only under the admin database (which you can access by entering `use admin`). The `top` command, for example, will output access details about all collections on the server.

```

> use admin
> db.runCommand("top")
{
  "totals" : {
    "note" : "all times in microseconds",
    "admin.system.roles" : {
      "total" : {
        "time" : 3666,
        "count" : 1
      },
      // etc
    }
  }
}

```

You can also list all commands that are currently available (let's switch back to the book database first because the admin database provides a different set of commands):

```

> use book
> db.listCommands()

```

When you run `listCommands()`, you may notice a lot of commands we've used already. In fact, you can execute many common commands through the `runCommand()` method, such as counting the number of phones. However, you may notice a slightly different output.

```

> db.runCommand({ "find" : "someCollection" })
{
  "cursor" : {
    "id" : NumberLong(0),
    "ns" : "book.someCollection",
    "firstBatch" : [ ]
  },
  "ok" : 1
}

```

Here, we see that this function returns an object containing a cursor and an `ok` field. That's because `db.phones.find()` is a wrapper function created for our convenience by the shell's JavaScript interface, whereas `runCommand()` is an operation executed on the server. Remember that we can play detective on how a function such as `find()` works by leaving off the calling parentheses.

```

> db.phones.find
function (query, fields, limit, skip, batchSize, options) {
  var cursor = new DBQuery(this._mongo,
    // other query parameters
    options || this.getQueryOptions());

  // some cursor-building logic

  return cursor;
}

```

So what about the DBQuery object? How much more can we find out about it?

```
> DBQuery
function DBQuery() {
  [native code]
}
```

Okay, looks like Mongo isn't going to reveal too much about that. No matter: this way of diving into function definitions is a great way to both explore Mongo conceptually and to get a better sense of what's happening inside of your queries and operations.

Diversion

We took this quick diversion into function definitions for two reasons:

- To drive home the idea that most of the magic you execute in the mongo console is executed on the server, not the client, which just provides some convenient wrapper functions.
- We can leverage the concept of executing server-side code for our own purposes to create something in MongoDB that's similar to the *stored procedures* we saw in PostgreSQL.

Any JavaScript function can be stored in a special collection named `system.js`. This is just a normal collection; you save the function by setting the name as the `_id` and a function object as the value.

```
> db.system.js.save({
  _id: 'getLast',
  value: function(collection) {
    return collection.find({}).sort({'_id':1}).limit(1)[0];
  }
})
```

Now you can use that function by loading it into the current namespace:

```
> use book
> db.loadServerScripts()
> getLast(db.phones).display
+1 800-5550010
```

Mapreduce (and Finalize)

Mapreduce operations are designed for performing computations over large datasets. Every mapreduce operation is split into two basic steps. First, a *map* step performs some series of filtering and/or sorting operation, winnowing the original dataset down into some subset. Then, a *reduce* step performs some kind of operation on that subset. An example mapreduce operation would be finding all baseball players in Major League history with the first

name Dave (the map step) and then finding the cumulative batting average for all of those Daves (the reduce step).

In MongoDB, the map step involves creating a mapper function that calls an `emit()` function. The benefit of this approach is you can emit more than once per document. The `reduce()` function accepts a single key and a list of values that were emitted to that key. Finally, Mongo provides an optional third step called `finalize()`, which is executed only once per mapped value after the reducers are run. This allows you to perform any final calculations or cleanup you may need.

Because we already know the basics of mapreduce, we'll skip the intro wading-pool example and go right to the high-dive. Let's generate a report that counts all phone numbers that contain the same digits for each country. First, we'll store a helper function that extracts an array of all distinct numbers (understanding how this helper works is not imperative to understanding the overall mapreduce).

`mongo/distinctDigits.js`

```
distinctDigits = function(phone){
  var number = phone.components.number + '',
      seen = [],
      result = [],
      i = number.length;

  while(i--){
    seen[+number[i]] = 1;
  }

  for (var i = 0; i < 10; i++) {
    if (seen[i]) {
      result[result.length] = i;
    }
  }

  return result;
}

db.system.js.save({_id: 'distinctDigits', value: distinctDigits})
```

Load the file in the mongo command line. If the file exists in the same directory you launched mongo from, you need only the filename; otherwise, a full path is required.

```
> load('distinctDigits.js')
```

Now we can get to work on the mapper. As with any mapreduce function, deciding what fields to map by is a crucial decision because it dictates the aggregated values that you return. Because our report is finding distinct numbers, the array of distinct values is one field. But because we also need

to query by country, that is another field. We add both values as a compound key: {digits : X, country : Y}.

Our goal is to simply count these values, so we emit the value 1 (each document represents one item to count). The reducer's job is to sum all those 1s together.

mongo/map1.js

```
map = function() {
  var digits = distinctDigits(this);
  emit({
    digits: digits,
    country: this.components.country
  }, {
    count : 1
  });
}
```

mongo/reduce1.js

```
reduce = function(key, values) {
  var total = 0;
  for (var i = 0; i < values.length; i++) {
    total += values[i].count;
  }
  return { count : total };
}

results = db.runCommand({
  mapReduce: 'phones',
  map:      map,
  reduce:   reduce,
  out:      'phones.report'
})
```

Because we set the collection name via the out parameter (out: 'phones.report'), you can query the results like any other. It's a materialized view that you can see in the show tables list.

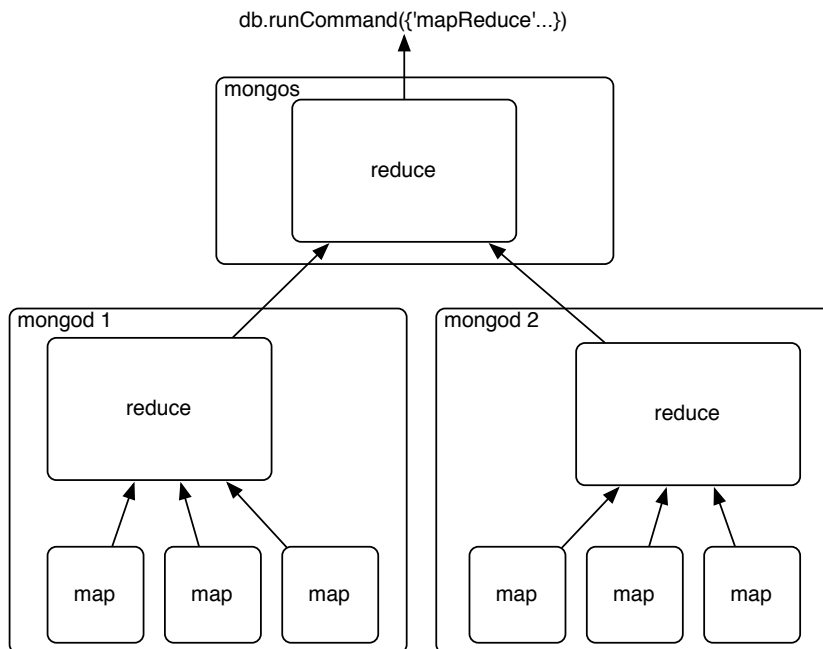
```
> db.phones.report.find({'_id.country' : 8})
{
  "_id" : { "digits" : [ 0, 1, 2, 3, 4, 5, 6 ], "country" : 8 },
  "value" : { "count" : 19 }
}
{
  "_id" : { "digits" : [ 0, 1, 2, 3, 5 ], "country" : 8 },
  "value" : { "count" : 3 }
}
{
  "_id" : { "digits" : [ 0, 1, 2, 3, 5, 6 ], "country" : 8 },
  "value" : { "count" : 48 }
}
```

```
{
  "_id" : { "digits" : [ 0, 1, 2, 3, 5, 6, 7 ], "country" : 8 },
  "value" : { "count" : 12 }
}
has more
```

Type it to continue iterating through the results. Note that the unique emitted keys are under the field `_ids`, and all of the data returned from the reducers is under the field `value`.

If you prefer that the mapreducer just output the results, rather than outputting to a collection, you can set the out value to `{ inline : 1 }`, but bear in mind that there is a limit to the size of a result you can output. As of Mongo 2.0, that limit is 16 MB.

Reducers can have either mapped (emitted) results or other reducer results as inputs. Why would the output of one reducer feed into the input of another if they are mapped to the same key? Think of how this would look if run on separate servers, as shown in the figure that follows.



Each server must run its own `map()` and `reduce()` functions and then push those results to be merged with the service that initiated the call, gathering them up. Classic divide and conquer. If we had renamed the output of the reducer

to total instead of count, we would have needed to handle both cases in the loop, as shown here:

`mongo/reduce2.js`

```
reduce = function(key, values) {
  var total = 0;
  for(var i = 0; i < values.length; i++) {
    var data = values[i];
    if('total' in data) {
      total += data.total;
    } else {
      total += data.count;
    }
  }
  return { total : total };
}
```

However, Mongo predicted that you might need to perform some final changes, such as renaming a field or some other calculations. If you really need the output field to be total, you can implement a `finalize()` function, which works the same way as the `finalize` function under `group()`.

Day 2 Wrap-Up

On Day 2, we've expanded our query power by including several aggregate queries: `count()`, `distinct()`, and topped off by `aggregate()`. To speed up the response time of these queries, we used MongoDB's indexing options. When more power is required, the ever-present `mapReduce()` is available.

Day 2 Homework

Find

1. Find a shortcut for admin commands.
2. Find the online documentation for queries and cursors.
3. Find the MongoDB documentation for mapreduce.
4. Through the JavaScript interface, investigate the code for three collections functions: `help()`, `findOne()`, and `stats()`.

Do

1. Implement a `finalize` method to output the count as the total.
2. Install a Mongo driver for a language of your choice, and connect to the database. Populate a collection through it, and index one of the fields.

Day 3: Replica Sets, Sharding, GeoSpatial, and GridFS

Mongo has a powerful ability to store and query data in a variety of ways. But then again, other databases can do those things, too. What makes document databases unique is their ability to efficiently handle arbitrarily nested, schemaless data documents. Thus far, we've run Mongo as a single server. But if you were to run Mongo in production, you'd want to run it as a cluster of machines, which would provide for much higher availability and enable you to replicate data across servers, shard collections into many pieces, and perform queries in parallel.



Replica Sets

Mongo was meant to scale out, not to run in standalone mode. It was built for data consistency and partition tolerance, but sharding data has a cost: If one part of a collection is lost, the whole thing is compromised. What good is querying against a collection of countries that contains only the Western Hemisphere or only Asia? Mongo deals with this implicit sharding weakness in a simple manner: duplication. You should rarely run a single Mongo instance in production and instead replicate the stored data across multiple services.

Rather than muck with our existing database, today we'll start from scratch and spawn a few new servers. Mongo's default port is 27017, so we'll start up each server on other ports. Remember that you must create the data directories first, so create three of them:

```
$ mkdir ./mongo1 ./mongo2 ./mongo3
```

Next, we'll fire up the Mongo servers. This time we'll add the `replSet` flag with the name *book* and specify the ports.

```
$ mongod --replSet book --dbpath ./mongo1 --port 27011
```


Open another terminal window and run the next command, which launches another server, pointing to a different directory, available on another port. Then open a third terminal to start the third server.

```
$ mongod --replSet book --dbpath ./mongo2 --port 27012
$ mongod --replSet book --dbpath ./mongo3 --port 27013
```

Notice that you get lots of this noise on the output, with error messages like this:

```
[initandlisten] Did not find local voted for document at startup
```

That's a good thing because we haven't yet initialized our replica set and Mongo is letting us know that. Fire up a mongo shell to one of the servers, and execute the `rs.initiate()` function.

```
$ mongo localhost:27011
> rs.initiate({
  _id: 'book',
  members: [
    { _id: 1, host: 'localhost:27011' },
    { _id: 2, host: 'localhost:27012' },
    { _id: 3, host: 'localhost:27013' }
  ]
})
> rs.status().ok
1
```

Notice we're using a new object called `rs` (replica set) instead of `db` (database). Like other objects, it has a `help()` method you can call. Running the `status()` command will let us know when our replica set is running, so just keep checking the status for completion before continuing. If you watch the three server outputs, you should see that one server outputs this line:

```
Member ... is now in state PRIMARY
```

And two servers will have the following output:

```
Member ... is now in state SECONDARY
```

PRIMARY will be the master server. Chances are, this will be the server on port 27011 (because it started first); however, if it's not, go ahead and fire up a console to the primary. Just insert any old thing on the command line, and we'll try an experiment.

```
> db.echo.insert({ say : 'HELLO!' })
```

After the insert, exit the console, and then let's test that our change has been replicated by shutting down the master node; pressing `Ctrl+C` is sufficient. If you watch the logs of the remaining two servers, you should see that one of the two has now been promoted to master (it will output the `Member ... is now in`

state PRIMARY line). Open a console into that machine (for us it was *localhost:27012*), and `db.echo.find()` should contain your value.

We'll play one more round of our console-shuffle game. Open a console into the remaining SECONDARY server by running `mongo localhost:27013`. Just to be sure, run the `isMaster()` function. Ours looked like this:

```
> db.isMaster().ismaster
false
> db.isMaster().primary
localhost:27012
```

In this shell, let's attempt to insert another value.

```
> db.echo.insert({ say : 'is this thing on?' })
WriteResult({ "writeError" : { "code" : 10107, "errmsg" : "not master" } })
```

This message is letting us know that we can neither write to a secondary node nor read directly from it. There is only one master per replica set, and you must interact with it. It is the gatekeeper to the set.

Replicating data has its own issues not found in single-source databases. In the Mongo setup, one problem is deciding who gets promoted when a master node goes down. Mongo deals with this by giving each mongod service a vote, and the one with the freshest data is elected the new master. Right now, you should still have two mongod services running. Go ahead and shut down the current master (aka primary node). Remember, when we did this with three nodes, one of the others just got promoted to be the new master. Now the last remaining node is implicitly the master.

Go ahead and relaunch the other servers and watch the logs. When the nodes are brought back up, they go into a recovery state and attempt to resync their data with the new master node. “Wait a minute!?” we hear you cry. “So, what if the original master had data that did not yet propagate?” Those operations are dropped. A write in a Mongo replica set isn't considered successful until most nodes have a copy of the data.

The Problem with Even Nodes

The concept of replication is easy enough to grasp: You write to one MongoDB server, and that data is duplicated across others within the replica set. If one server is unavailable, then one of the others can be promoted and serve requests. But a server can be unavailable in more ways than a server crash. Sometimes, the network connection between nodes is down (such as a network *partition*, thinking back to the P in CAP). In that case, Mongo dictates that *the majority of nodes that can still communicate now constitute the network*.

MongoDB expects an odd number of total nodes in the replica set. Consider a five-node network, for example. If connection issues split it into a three-node fragment and a two-node fragment, the larger fragment has a clear majority and can elect a master and continue servicing requests. With no clear majority, a quorum couldn't be reached.

To see why an odd number of nodes is preferred, consider what might happen to a four-node replica set. Say a network partition causes two of the servers to lose connectivity from the other two. One set will have the original master, but because it can't see a *clear majority* of the network, the master steps down. The other set will similarly be unable to elect a master because it, too, can't communicate with a clear majority of nodes. Both sets are now unable to process requests and the system is effectively down. Having an odd number of total nodes would have made this particular scenario—a fragmented network where each fragment has less than a clear majority—less likely to occur.

Some databases (such as CouchDB) are built to allow multiple masters, but Mongo is not, and so it isn't prepared to resolve data updates between them. MongoDB deals with conflicts between multiple masters by simply not allowing them.

Because it's a CP system, Mongo always knows the most recent value; the client needn't decide. Mongo's concern is strong consistency on writes, and preventing a multimaster scenario is not a bad method for achieving it.

Voting and Arbiters

You may not always want to have an odd number of servers replicating data. In that case, you can either launch an arbiter (generally recommended) or increase voting rights on your servers (generally not recommended). In Mongo, an arbiter is a voting but nonreplicating server in the replica set. You launch it just like any other server but on configuration set a flag, like this: `{_id: 3, host: 'localhost:27013', arbiterOnly : true}`. Arbiters are useful for breaking ties, like the U.S. Vice President in the Senate. By default, each mongod instance has a single vote.

Sharding

One of the core goals of Mongo is to provide safe and quick handling of very large datasets. The clearest method of achieving this is through horizontal sharding by value ranges—or just *sharding* for brevity. Rather than a single server hosting all values in a collection, some range of values is split, or *sharded*, onto other servers. For example, in our phone numbers collection, we may put all phone numbers less than 1-500-000-0000 onto Mongo server

A and put numbers greater than or equal to 1-500-000-0001 onto server B. Mongo makes this easier by autosharding, managing this division for you.

Let's launch a couple of (nonreplicating) mongod servers. Like replica sets, there's a special parameter necessary to be considered a shard server (which just means this server is capable of sharding).

```
$ mkdir ./mongo4 ./mongo5
$ mongod --shardsvr --dbpath ./mongo4 --port 27014
$ mongod --shardsvr --dbpath ./mongo5 --port 27015
```

Now you need a server to actually keep track of your keys. Imagine you created a table to store city names alphabetically. You need some way to know that, for example, cities starting with A through N go to server mongo4 and O through Z go to server mongo5. In Mongo, you create a *config server* (which is just a regular mongod) that keeps track of which server (mongo4 or mongo5) owns what values. You'll need to create and initialize a second replica set for the cluster's configuration (let's call it configSet).

```
$ mkdir ./mongoconfig
$ mongod --configsvr --replSet configSet --dbpath ./mongoconfig --port 27016
```

Now enter the Mongo shell for the config server by running `mongo localhost:27016` and initiate the config server cluster (with just one member for this example):

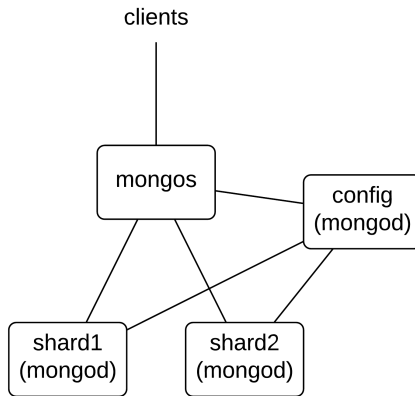
```
> rs.initiate({
  _id: 'configSet',
  configsvr: true,
  members: [
    {
      _id: 0,
      host: 'localhost:27016'
    }
  ]
})
{ "ok" : 1 }
> rs.status().ok
1
```

Finally, you need to run yet another server called mongos, which is the single point of entry for our clients. The mongos server will connect to the mongoconfig config server to keep track of the sharding information stored there. You point mongos to the replSet/server:port with the `--configdb` flag.

```
$ mongos --configdb configSet/localhost:27016 --port 27020
```

A neat thing about mongos is that it is a lightweight clone of a full mongod server. Nearly any command you can throw at a mongod you can throw at a

mongos, which makes it the perfect go-between for clients to connect to multiple sharded servers. The following picture of our server setup may help.



Now let's jump into the mongos server console in the admin database by running `mongo localhost:27020/admin`. We're going to configure some sharding.

```

> sh.addShard('localhost:27014')
{ "shardAdded" : "shard0000", "ok" : 1 }
> sh.addShard('localhost:27015')
{ "shardAdded" : "shard0001", "ok" : 1 }

```

With that setup, now we have to give it the database and collection to shard and the field to shard by (in our case, the city name).

```

> db.runCommand({ enablesharding : "test" })
{ "ok" : 1 }
> db.runCommand({ shardcollection : "test.cities", key : {name : 1} })
{ "collectionsharded" : "test.cities", "ok" : 1 }

```

mongos vs. mongoconfig

You may wonder why Mongo separates the config server and the mongos *point of entry* into two different servers. In production environments, they will generally live on different physical servers. The config server (which can itself be replicated across multiple servers) manages the sharded information for other sharded servers, while mongos will likely live on your local application server where clients can easily connect (without needing to manage which shards to connect to).

With all that setup out of the way, let's load some data. If you download the book code, you'll find a 12 MB data file named `mongoCities100000.json` that contains data for every city in the world with a population of more than 1,000 people. Download that file, and run the following import script that imports the data into your mongos server:

```
$ mongoimport \
  --host localhost:27020 \
  --db test \
  --collection cities \
  --type json \
  mongoCities100000.json
```

If the import is successful, you should see imported 99838 documents in the output (not quite 100,000 cities as the filename would suggest, but pretty close).

GeoSpatial Queries

Mongo has a neat trick built into it. Although we've focused on server setups today, no day would be complete without a little bit of razzle-dazzle, and that's Mongo's ability to quickly perform geospatial queries. First, connect to the mongos sharded server.

```
$ mongo localhost:27020
```

The core of the geospatial secret lies in indexing. It's a special form of indexing geographic data called *geohash* that not only finds values of a specific value or range quickly but finds nearby values quickly in ad hoc queries. Conveniently, at the end of our previous section, we installed a lot of geographic data. So to query it, step 1 is to index the data on the location field. The *2d* index must be set on any two value fields, in our case a hash (for example, { longitude:1.48453, latitude:42.57205 }), but it could easily have been an array (for example [1.48453, 42.57205]).

```
> db.cities.ensureIndex({ location : "2d" })
```

Now, we can use an aggregation pipeline (think back to Day 2) to assemble a list of all cities close to Portland, OR sorted in descending order by population (displaying the name of the city, the population, and the distance from the 45.52/-122.67 latitude/longitude point).

```
> db.cities.aggregate([
  {
    $geoNear: {
      near: [45.52, -122.67],
      distanceField: 'dist'
    }
  },
  {
    $sort: {
      population: -1
    }
  },
  {
```

```

    $project: {
      _id: 0,
      name: 1,
      population: 1,
      dist: 1
    }
  }
})
{ "name" : "Portland", "population" : 540513, "dist" : 0.007103984797274343 }
{ "name" : "Vancouver", "population" : 157517, "dist" : 0.11903458741054997 }
{ "name" : "Salem", "population" : 146922, "dist" : 0.6828926855663344 }
{ "name" : "Gresham", "population" : 98851, "dist" : 0.2395159760851125 }
// many others

```

As you can see, Mongo's aggregation API provides a very nice interface for working with schemaless geospatial data. We've only scratched the surface here (no pun intended), but if you're interested in exploring the full potential of MongoDB, we strongly encourage you to dig more deeply.⁴

GridFS

One downside of a distributed system can be the lack of a single coherent filesystem. Say you operate a website where users can upload images of themselves. If you run several web servers on several different nodes, you must manually replicate the uploaded image to each web server's disk or create some alternative central system. Mongo handles this scenario using a distributed filesystem called GridFS.

Mongo comes bundled with a command-line tool for interacting with GridFS. The great thing is that you don't have to set up anything special to use it. If you list the files in the mongos managed shards using the command `mongofiles`, you get an empty list.

```
$ mongofiles -h localhost:27020 list
```

```
connected to: localhost:27020
```

But upload any file...

```

$ echo "here's some file data" > just-some-data.txt
$ mongofiles -h localhost:27020 put just-some-data.txt
2017-05-11T20:03:32.272-0700    connected to: localhost:27020
added file: my-file.txt

```

...and *voilà!* If you list the contents of `mongofiles`, you'll find the uploaded name.

```
$ mongofiles -h localhost:27020 list
```

```

2017-05-11T20:04:39.019-0700    connected to: localhost:27020
just-some-data.txt             22

```

4. <https://docs.mongodb.com/manual/core/2dsphere>

Back in your mongo console, you can see the collections Mongo stores the data in.

```
> show collections
cities
fs.chunks
fs.files
```

Because they're just plain old collections, they can be replicated or queried like any other. Here we'll look up the filename of the text file we imported.

```
> db.fs.files.find()[0].filename
just-some-data.txt
```

Day 3 Wrap-Up

This wraps up our investigation of MongoDB. Today we focused on how Mongo enhances data durability with replica sets and supports horizontal scaling with sharding. We looked at good server configurations and how Mongo provides the mongos server to act as a relay for handling autosharding between multiple nodes. Finally, we toyed with some of Mongo's built-in tools, such as geospatial queries and GridFS.

Day 3 Homework

Find

1. Read the full replica set configuration options in the online docs.
2. Find out how to create a spherical geo index.

Do

1. Mongo has support for bounding shapes (namely, squares and circles). Find all cities within a 50-mile radius around the center of London.⁵
2. Run six servers: three servers in a replica set, and each replica set is one of two shards. Run a config server and mongos. Run GridFS across them (this is the final exam).

Wrap-Up

We hope this taste of MongoDB has piqued your fancy and showed you how it earns the moniker of the “humongous” database. We covered a lot in a single chapter, but as usual, we only clawed at the surface.

5. <https://docs.mongodb.com/manual/reference/operator/query/geoWithin/>

Mongo's Strengths

Mongo's primary strength lies in its ability to handle huge amounts of data (and huge amounts of requests) by replication and horizontal scaling. But it also has an added benefit of a very flexible data model. You don't ever need to conform to a schema and can simply nest any values you would generally join using SQL in an RDBMS.

Finally, MongoDB was built to be easy to use. You may have noticed the similarity between Mongo commands and SQL database concepts (minus the server-side joins). This is not by accident and is one reason Mongo has so much mind share amongst people who have defected from the relational database camp. It's different enough to scratch a lot of developer itches but not so different that it becomes a weird and scary monster.

Mongo's Weaknesses

Mongo encourages denormalization of schemas (by not having any) and that can be a bit too much for some to swallow. Some developers find the cold, hard constraints of a relational database reassuring. It can be dangerous to insert any old value of any type into any collection. A single typo can cause hours of headache if you don't think to look at field names and collection names as a possible culprit. Mongo's flexibility is generally not important if your data model is already fairly mature and locked down.

Because Mongo is focused on large datasets, it works best in large clusters, which can require some effort to design and manage. Unlike some clustered databases where adding new nodes is a transparent and relatively painless process, setting up a Mongo cluster requires a little more forethought.

Parting Thoughts

Mongo is an excellent choice if you are currently using a relational database to store your data using ORM-style patterns. We often recommend it to developers steeped in frameworks such as Ruby on Rails and Django because they can then perform validations and field management through the models at the application layer and because schema migrations become a thing of the past (for the most part). Adding new fields to a document is as easy as adding a new field to your data model, and Mongo will happily accept the new terms. We find Mongo to be a much more natural answer to many common problem scopes for application-driven datasets than relational databases.